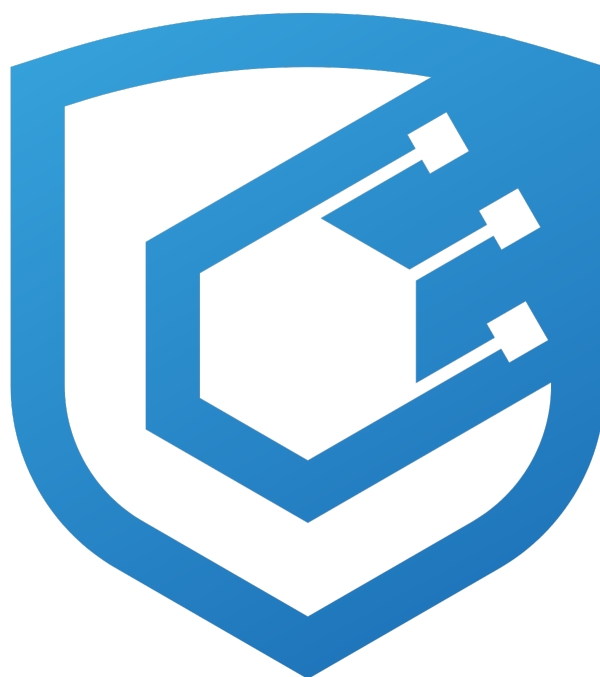




MetaMask Compliance Veda Adapter Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Kage](#)

[Flint](#)

October 5, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	4
7.1	Low Risk	4
7.1.1	ComplianceVedaAdapter::_executeWithdrawByDelegation withdraws as the Teller's msg.sender, so the beneficiary's principal checkpoint is skipped.	4
7.2	Informational	7
7.2.1	ComplianceVedaAdapter does not validate the enforcer whose terms it parses	7
7.2.2	ComplianceVedaLending negative tests accept any revert and its fork state is not reproducible	7
7.2.3	DeployComplianceVedaAdapter script does not validate the adapter's immutable target configuration	7
7.2.4	ComplianceVedaAdapter NatSpec omits the roles required for withdrawals	8
7.2.5	AllowedMethodsEnforcer guidance omits the delegated ERC20 transfer selector	9
7.2.6	ComplianceVedaAdapter::depositByDelegation NatSpec recommends a 0.1–0.5% mint tolerance that the Teller's configurable share premium can invalidate	10
7.2.7	The documented --skip-simulation workaround does not resolve deployment failure	11
7.2.8	Unclear compliance rejection semantics can lead to duplicate deposits on retry	12

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

`ComplianceVedaAdapter` connects MetaMask's delegation framework to a compliance-enabled Veda BoringVault. It lets a delegated operator execute deposits and withdrawals for a user's DeleGator account without the user directly approving or calling the Veda contracts. The adapter is permissionless to call; authorization comes from the delegation chain, its caveats, and—for deposits—the Veda Teller's compliance check.

5 Audit Scope

The audit scope was focused on [PR205](#) limited to:

```
script/DeployComplianceVedaAdapter.s.sol  
src/helpers/ComplianceVedaAdapter.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the [Compliance Veda Adapter](#) code provided by [MetaMask](#). The findings consist of 1 Low severity issue with the remainder being informational.

Summary

Project Name	Compliance Veda Adapter
Repository	delegation-framework
Commit	bba859626789...
Fix Commit	9a8fa2bffe3b...
Audit Timeline	Sep 23rd - Sep 24th, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	1	0	1
Informational	8	6	2
Gas Optimizations	0	0	0
Total	9	6	3

Summary of Findings

[L-1] <code>ComplianceVedaAdapter::_executeWithdrawByDelegation</code> withdraws as the Teller's <code>msg.sender</code> , so the beneficiary's principal checkpoint is skipped.	Acknowledged
[I-1] <code>ComplianceVedaAdapter</code> does not validate the enforcer whose terms it parses	Acknowledged
[I-2] <code>ComplianceVedaLending</code> negative tests accept any revert and its fork state is not reproducible	Resolved
[I-3] <code>DeployComplianceVedaAdapter</code> script does not validate the adapter's immutable target configuration	Acknowledged
[I-4] <code>ComplianceVedaAdapter</code> <code>NatSpec</code> omits the roles required for withdrawals	Resolved
[I-5] <code>AllowedMethodsEnforcer</code> guidance omits the delegated ERC20 transfer selector	Resolved
[I-6] <code>ComplianceVedaAdapter::depositByDelegation</code> <code>NatSpec</code> recommends a 0.1–0.5% mint tolerance that the Teller's configurable share premium can invalidate	Resolved
[I-7] The documented <code>–skip-simulation</code> workaround does not resolve deployment failure	Resolved
[I-8] Unclear compliance rejection semantics can lead to duplicate deposits on retry	Resolved

7 Findings

7.1 Low Risk

7.1.1 ComplianceVedaAdapter::_executeWithdrawByDelegation **withdraws as the Teller's msg.sender, so the beneficiary's principal checkpoint is skipped.**

Description: The Monad compliance Teller (Base V0.3) keeps a per-user principal history that is appended on deposits and withdrawals. Deposits are checkpointed against the share recipient `to`, withdrawals are checkpointed against `msg.sender`:

```
// TellerWithMultiAssetSupport.sol (contract address 0xB0025a2eBc0474d4F28E975F0D3E70471246ebae)
function deposit(...) external payable virtual requiresAuth nonReentrant returns (uint256 shares) {
    ...
    _checkpointPrincipalAtRate(to, shares, true, rate, rate);           // @audit recorded for the
    ↪ beneficiary
}

function _withdraw(ERC20 withdrawAsset, uint256 shareAmount, uint256 minimumAssets, address to)
    ↪ internal virtual returns (uint256 assetsOut) {
    ...
    _checkpointPrincipalAtRate(msg.sender, shareAmount, false, rate, rate); // @audit recorded for
    ↪ msg.sender, not `to`
}
```

Through the adapter, `msg.sender` is the adapter and `to` is the root delegator:

```
// src/helpers/ComplianceVedaAdapter.sol:445
// @audit adapter is msg.sender; the Teller checkpoints the adapter, not rootDelegator_
uint256 assetsOut_ = teller.withdraw(address(depositToken), shareAmount_, _minimumAssets,
    ↪ rootDelegator_);
```

The adapter never deposits for itself, so it has no history and the library discards the record:

```
// TellerWithMultiAssetSupportLib.sol:73
if (!isDeposit && len == 0) return; // @audit user == adapter, len == 0 -> withdrawal silently dropped
```

The library's own comment says correctness "relies on shares being non-transferable". Adapter is exactly the shares-moved-then-burned-by-a-router case it warns about.

Teller already provides `checkpointQueueWithdrawal(user, shares)` (gated to `QUEUE_ROLE`, role 32 on Monad) for trusted routers or queues that bypass the normal withdrawal checkpoint, but the adapter does not call it. Teller's Withdraw event consequently identifies the adapter as `user = msg.sender` and the beneficiary as `recipient = to`; consumers that interpret `user` as the beneficial owner must account for this routed flow.

Impact: Every adapter withdrawal leaves the user's on-chain principal history one-sided: `cumulativeDeposits` grows, while the corresponding `cumulativeWithdrawals` entry is omitted. Three round-trips of the same 1,000 mUSD leave the user with zero shares, their mUSD back, and approximately 3,000 mUSD of recorded net principal (refer PoC).

The history is append-only, but the omission is not wholly irreparable: an authorized caller can later append compensating withdrawals through `checkpointQueueWithdrawal`, and an off-chain consumer can reconstruct routed withdrawals from events. A compensating checkpoint uses the rate and timestamp at the time it is submitted, however, so it cannot recreate the exact historical checkpoint at the original withdrawal's rate and time.

Nothing on-chain reads the history today, `IncentivePool::processRewards` pays a backend-signed `cumulativeOwed`, and `claimRewards` is not currently a public capability on Monad. Corrupting the history therefore does not by itself cause a reward payment. The confirmed impact is incorrect principal-accounting history and the loss of exact historical fidelity.

Proof of Concept: Runs against the PR's own Monad fork `setUp()`. Add the struct, interface and test below to `test/helpers/ComplianceVedaLending.t.sol`

Run the following:

```
MONAD_RPC_URL=https://rpc.monad.xyz forge test --mt
↳ test_POC_adapterWithdrawSkipsUserPrincipalCheckpoint -vvv
```

```
struct PrincipalCheckpoint { uint48 timestamp; uint104 cumulativeDeposits; uint104
↳ cumulativeWithdrawals; uint256 vaultSharePrice; }

interface ITellerPrincipalHistory {
    function getPrincipalHistoryPaginated(address user, uint256 startIndex, uint256 length)
        external view returns (PrincipalCheckpoint[] memory checkpoints, uint256 totalLength);
}

event Withdraw(address indexed asset, uint256 shareAmount, address indexed user, address indexed
↳ recipient);

/// @notice Adapter withdrawals are checkpointed against the adapter (msg.sender), not the root
↳ delegator.
/// 1. Alice deposits 1,000 mUSD via the adapter -> Teller appends a deposit checkpoint for
↳ Alice
/// 2. Alice withdraws all shares via the adapter -> Teller checkpoints the adapter, which has
↳ no
/// history, so the record is dropped; `Withdraw` event names the adapter as `user`
/// 3. Alice's history still shows cumulativeWithdrawals == 0 although she holds zero shares
/// 4. Three round-trips with the same 1,000 mUSD leave ~3,000 mUSD of recorded net principal
/// 5. Contrast: the same deposit + withdraw done directly by Alice records both sides
function test_POC_adapterWithdrawSkipsUserPrincipalCheckpoint() public {
    ITellerPrincipalHistory teller_ = ITellerPrincipalHistory(address(TELLER));
    address alice_ = address(users.alice.deleGator);
    uint256 musdBefore_ = MUSD.balanceOf(alice_);

    _depositViaAdapter(DEPOSIT_AMOUNT, 1000, block.timestamp + 30 minutes);
    (PrincipalCheckpoint[] memory cps_, uint256 len_) = teller_.getPrincipalHistoryPaginated(alice_, 0,
↳ 10);
    assertEq(len_, 1, "deposit checkpoint recorded for Alice");
    assertApproxEqRel(cps_[0].cumulativeDeposits, DEPOSIT_AMOUNT, 0.001e18);
    assertEq(cps_[0].cumulativeWithdrawals, 0);

    uint256 shares_ = BORING_VAULT.balanceOf(alice_);
    Delegation[] memory withdrawChain_ = _createDelegationChain(address(BORING_VAULT), shares_, 1001);
    vm.expectEmit(true, false, true, true, address(TELLER));
    emit Withdraw(address(MUSD), shares_, address(adapter), alice_);
    vm.prank(address(users.bob.deleGator));
    adapter.withdrawByDelegation(withdrawChain_, 0);
    assertEq(BORING_VAULT.balanceOf(alice_), 0, "Alice holds no shares");

    (cps_, len_) = teller_.getPrincipalHistoryPaginated(alice_, 0, 10);
    assertEq(len_, 1, "no withdrawal checkpoint appended for Alice");
    assertEq(cps_[0].cumulativeWithdrawals, 0, "Alice cumulativeWithdrawals still zero");
    (, uint256 adapterLen_) = teller_.getPrincipalHistoryPaginated(address(adapter), 0, 10);
    assertEq(adapterLen_, 0, "adapter has no principal history");

    for (uint256 i = 1; i < 3; ++i) {
        _depositViaAdapter(DEPOSIT_AMOUNT, 1000 + 10 * i, block.timestamp + 30 minutes + i);
        shares_ = BORING_VAULT.balanceOf(alice_);
        withdrawChain_ = _createDelegationChain(address(BORING_VAULT), shares_, 1001 + 10 * i);
        vm.prank(address(users.bob.deleGator));
        adapter.withdrawByDelegation(withdrawChain_, 0);
    }
    (cps_, len_) = teller_.getPrincipalHistoryPaginated(alice_, 0, 10);
    assertEq(len_, 3, "three deposit checkpoints, zero withdrawal checkpoints");
    uint256 netPrincipal_ = uint256(cps_[2].cumulativeDeposits) -
↳ uint256(cps_[2].cumulativeWithdrawals);
```

```

assertApproxEqRel(netPrincipal_, 3 * DEPOSIT_AMOUNT, 0.01e18, "phantom net principal ~3,000 mUSD");
assertEq(BORING_VAULT.balanceOf(alice_), 0);
assertApproxEqRel(MUSD.balanceOf(alice_), musdBefore_, 0.001e18, "Alice's mUSD is back");

uint256 deadline_ = block.timestamp + 30 minutes;
bytes memory sig_ = _signCompliance(alice_, alice_, DEPOSIT_AMOUNT, deadline_,
    ↪ COMPLIANCE_SIGNER_KEY);
vm.startPrank(alice_);
MUSD.approve(address(BORING_VAULT), DEPOSIT_AMOUNT);
uint256 directShares_ = TELLER.deposit(
    IComplianceVedaTeller.DepositParams(address(MUSD), DEPOSIT_AMOUNT, 0), alice_, address(0),
    IComplianceVedaTeller.ComplianceData(deadline_, sig_)
);
TELLER.withdraw(address(MUSD), directShares_, 0, alice_);
vm.stopPrank();
(cps_, len_) = teller_.getPrincipalHistoryPaginated(alice_, 0, 10);
assertEq(len_, 5, "direct path appended deposit AND withdrawal checkpoints");
assertApproxEqRel(cps_[4].cumulativeWithdrawals, DEPOSIT_AMOUNT, 0.001e18, "direct withdrawal
    ↪ recorded");
netPrincipal_ = uint256(cps_[4].cumulativeDeposits) - uint256(cps_[4].cumulativeWithdrawals);
assertApproxEqRel(netPrincipal_, 3 * DEPOSIT_AMOUNT, 0.01e18, "adapter round-trips left phantom
    ↪ principal");
}

```

Recommended Mitigation: Record the beneficiary's withdrawal checkpoint right after the Teller withdrawal, using the hook the Teller already exposes for routers. Veda must grant the adapter QUEUE_ROLE (role 32).

```

// src/helpers/interfaces/IComplianceVedaTeller.sol
function withdraw(address withdrawAsset, uint256 shareAmount, uint256 minimumAssets, address to)
    ↪ external returns (uint256 assetsOut);
+
+ /// @notice Records a withdrawal principal checkpoint on behalf of `user`. Callable by QUEUE_ROLE.
+ function checkpointQueueWithdrawal(address user, uint256 shares) external;

```

```

// src/helpers/ComplianceVedaAdapter.sol - _executeWithdrawByDelegation
uint256 assetsOut_ = teller.withdraw(address(depositToken), shareAmount_, _minimumAssets,
    ↪ rootDelegator_);
+ // The Teller checkpointed msg.sender (this adapter); record the withdrawal against the
    ↪ beneficial owner
+ teller.checkpointQueueWithdrawal(rootDelegator_, shareAmount_);
    emit WithdrawExecuted(rootDelegator_, address(depositToken), shareAmount_, assetsOut_);

```

A cleaner alternative could be, Veda exposes a router-aware withdrawal that burns the router's shares while checkpointing and emitting for the beneficiary atomically, so no second call and no extra role are needed.

Metamask:

Acknowledged. The getPrincipalHistoryPaginated view is not consumed by any on-chain logic or by our off-chain reward systems today, so this does not affect user balances or rewards.

7.2 Informational

7.2.1 ComplianceVedaAdapter **does not validate the enforcer whose terms it parses**

Description: The adapter documents that `_delegations[0].caveats[0]` must use `ERC20TransferAmountEnforcer` with 52-byte `abi.encodePacked(token, amount)` terms, but `_parseERC20TransferTerms` checks only that the payload is at least 52 bytes and never checks the caveat's enforcer:

```
function _parseERC20TransferTerms(bytes calldata _terms) private pure returns (uint256 amount_) {
    if (_terms.length < 52) revert InvalidTermsLength();
    amount_ = uint256(bytes32(_terms[20:52]));
}
```

The expected enforcer independently requires exactly 52 bytes and enforces the transfer target and amount. However, if the operator mistakenly signs a leaf whose first caveat uses another enforcer that permits the inner transfer, the adapter still interprets bytes `[20:52]` of that enforcer's unrelated terms as the amount. A mismatched deposit is rejected by the compliance signature's amount binding; withdrawals have no equivalent check.

Recommended Mitigation: Consider storing the expected `ERC20TransferAmountEnforcer` as an immutable and validate both the enforcer and exact terms length before parsing.

Metamask:

Acknowledged. In practice, our backend always constructs delegation leaves with the documented `ERC20TransferAmountEnforcer`, and incorrect enforcers or malformed terms would cause the delegation chain or adapter execution to fail.

7.2.2 ComplianceVedaLending **negative tests accept any revert and its fork state is not reproducible**

Description: Five negative tests use `vm.expectRevert()` without an expected selector:

```
// test/helpers/ComplianceVedaLending.t.sol:160, 171, 182, 193 and 204
vm.expectRevert();
```

The replayed, expired, wrong-signer, and wrong-recipient cases should specifically revert with `TellerWithMultiAssetSupport__ComplianceCheckFailed()`, while the excessive minimum-mint case should revert with `TellerWithMultiAssetSupport__MinimumMintNotMet()`. Bare assertions can pass after a regression that makes the call fail earlier for an unrelated reason.

The suite also forks the latest Monad block and depends on fixed contract and whale addresses plus `authorityOwner == tellerOwner`. This is useful as a latest-state integration check, but it is not deterministic: unrelated live governance, balances, or configuration changes can break or change the test without a repository change.

Recommended Mitigation:

- Consider asserting `ComplianceCheckFailed` in the four compliance-negative tests and `MinimumMintNotMet` in the minimum-mint test.
- Consider pinning the fork block used by deterministic CI tests; optionally retain a separate latest-block smoke test.

Metamask:

Fixed in commit [686f0ab](#). For the fork block we use latest so we don't need a historical node, and it does not affect the quality of the code.

Cyfrin: Verified.

7.2.3 DeployComplianceVedaAdapter **script does not validate the adapter's immutable target configuration**

Description: `DeployComplianceVedaAdapter` reads five addresses from the environment and immediately deploys the adapter. The adapter constructor rejects zero addresses and calls `depositTo-`

ken.forceApprove(boringVault, type(uint256).max), but neither the script nor the constructor verifies that:

- the configured dependencies are contracts;
- vedaTeller.vault() equals boringVault; or
- depositToken is enabled for both deposits and withdrawals on that Teller.

Consequently, a set of individually valid but inconsistent addresses can deploy successfully. For example, pairing a real Teller with a different BoringVault still lets the constructor approve that vault, but the Teller's actual vault will not have the allowance needed to pull the adapter's tokens. Since the Teller, vault, and token are immutable, the deployment cannot be corrected in place.

```
// script/DeployComplianceVedaAdapter.s.sol
address deployed = address(
    new ComplianceVedaAdapter{ salt: salt }(
        vedaAdapterOwner, delegationManager, boringVault, vedaTeller, depositToken
    )
);
```

Recommended Mitigation: Before broadcasting, consider validating the target contracts and their relationships:

```
+interface ITellerConfig {
+    function vault() external view returns (address);
+    function assetData(address)
+        external
+        view
+        returns (bool allowDeposits, bool allowWithdraws, uint16 sharePremium);
+}

contract DeployComplianceVedaAdapter is Script {
    function setUp() public {

+        require(delegationManager.code.length != 0, "invalid delegation manager");
+        require(boringVault.code.length != 0, "invalid vault");
+        require(vedaTeller.code.length != 0, "invalid teller");
+        require(depositToken.code.length != 0, "invalid token");
+        require(ITellerConfig(vedaTeller).vault() == boringVault, "teller/vault mismatch");
+
+        (bool allowDeposits, bool allowWithdraws,) = ITellerConfig(vedaTeller).assetData(depositToken);
+        require(allowDeposits && allowWithdraws, "token not enabled");
    }
}
```

Metamask:

Acknowledged. All addresses (Teller, vault, token, delegation manager) are currently verified manually as part of our deployment process.

7.2.4 ComplianceVedaAdapter NatSpec omits the roles required for withdrawals

Description: The adapter documentation mentions allowlistedRouterRole only in the context of minting shares during deposits and then states that withdrawals are not compliance-gated:

```
// src/helpers/ComplianceVedaAdapter.sol:40-41
* - When `allowlistedRouterRole` is enabled on the Teller, this adapter must hold that role so it
↪ can
* mint shares to `to != msg.sender`. Withdrawals are not compliance-gated.
```

Although withdrawals do not require a compliance signature, the production Teller applies two other role checks to the adapter flow:

Flow	Check
Deposit and withdrawal	The Teller's <code>_checkRecipient(to)</code> sees <code>msg.sender = adapter</code> and <code>to = rootDelegator</code>
Withdrawal share pull	<code>BoringVault.transfer(adapter, shares)</code> invokes the Teller hook, which accepts <code>transferAllowed</code>

The router role is therefore required for both directions, not only deposits. The transfer role is additionally required for ordinary withdrawals because users and their operators are not generally expected to hold it. Both Teller selectors are currently public capabilities, so the adapter does not need a separate capability-bearing role under the live configuration; deployment should nevertheless verify that they remain callable.

Recommended Mitigation: Consider documenting both role requirements and distinguish compliance signatures from the Teller's routing and transfer restrictions:

```
- *      - When `allowlistedRouterRole` is enabled on the Teller, this adapter must hold that role so it
↳ can
- *      mint shares to `to != msg.sender`. Withdrawals are not compliance-gated.
+ *      - Veda RolesAuthority prerequisites:
+ *      - `allowlistedRouterRole` must permit this adapter to route both deposits and withdrawals to
↳ the
+ *      root delegator, because the Teller sees `to != msg.sender` in both flows.
+ *      - When `transferAllowedRole` is enabled, withdrawals must satisfy that role during the
↳ delegated
+ *      vault-share transfer. Granting it to this adapter supports all users.
+ *      - The Teller's deposit and withdraw selectors must be public or otherwise callable by this
↳ adapter.
+ *      Withdrawals do not require a compliance signature, but these role checks still apply.
```

Metamask:

Fixed in commit [9578aff](#)

Cyfrin: Verified

7.2.5 AllowedMethodsEnforcer guidance omits the delegated ERC20 transfer selector

Description: The delegation-flow documentation in `ComplianceVedaAdapter.sol:43-51` recommends that the operator's redelegation include:

```
* - Allowed methods enforcer limiting which functions can be called
```

The guidance could be clarified to identify the function selector evaluated by this caveat. The adapter entry point, the delegated token transfer, and the subsequent Teller call are separate executions; only the token transfer is passed to `DelegationManager.redeemDelegations`.

For deposits, the adapter constructs the following execution at `ComplianceVedaAdapter.sol:399-402`:

```
executionCallDatas_[0] =
    ExecutionLib.encodeSingle(address(depositToken), 0, abi.encodeCall(IERC20.transfer, (address(this),
↳ amount_)));

delegationManager.redeemDelegations(permissionContexts_, encodedModes_, executionCallDatas_);
```

For withdrawals, the transfer targets the vault share token instead, at `ComplianceVedaAdapter.sol:439-442`:

```
executionCallDatas_[0] =
    ExecutionLib.encodeSingle(boringVault, 0, abi.encodeCall(IERC20.transfer, (address(this),
↳ shareAmount_)));

delegationManager.redeemDelegations(permissionContexts_, encodedModes_, executionCallDatas_);
```

The DelegationManager passes this encoded execution to the caveat hooks. AllowedMethodsEnforcer._validateMethods decodes it and compares the first four bytes of the inner call data against the selectors encoded in its terms:

```
(, , bytes calldata callData_) = _executionCallData.decodeSingle();
bytes4 targetSig_ = bytes4(callData_[0:4]);
```

Consequently, an AllowedMethods caveat included in either route needs to allow IERC20.transfer.selector (0xa9059cbb), for example through abi.encodePacked(IERC20.transfer.selector). Allowing only depositByDelegation.selector, withdrawByDelegation.selector, or a Teller selector would not match this execution. The Teller call occurs after delegation redemption returns and is outside these caveat hooks.

Both routes use the same transfer selector, so an allowed-methods restriction alone does not distinguish deposits from withdrawals. Their transfer targets differ: depositToken for deposits and boringVault for withdrawals. The required ERC20TransferAmountEnforcer already validates both the token target and IERC20.transfer.selector; the additional AllowedMethods caveat is redundant for these checks.

Impact: Specifying the inner transfer selector would make the delegation-construction guidance easier to follow and avoid configuring a caveat that rejects the intended execution. It would also clarify that the caveat constrains the delegated token transfer, rather than selecting which adapter or Teller entry point may be called.

Recommended Mitigation: Consider identifying the selector, transfer targets, and execution boundary in the allowed-methods guidance, for example:

```
* - If an AllowedMethodsEnforcer caveat is included, allow
*   IERC20.transfer.selector (0xa9059cbb). It evaluates the preliminary
*   token transfer, not the adapter entry point or subsequent Teller call.
* - The required ERC20TransferAmountEnforcer already validates this
*   selector and the token target: depositToken for deposits and
*   boringVault for withdrawals.
```

MetaMask: Fixed in commit [83d430e](#).

Cyfrin: Verified.

7.2.6 ComplianceVedaAdapter::depositByDelegation NatSpec recommends a 0.1–0.5% mint tolerance that the Teller's configurable share premium can invalidate

Description: The deposit documentation assumes shares are always minted at fair value:

```
// src/helpers/ComplianceVedaAdapter.sol:236-238
*   bound. The Veda vault conversion is always at fair value; rate drift from yield streaming
*   is negligible. A tolerance of 0.1-0.5% is recommended. // @audit ignores
↳ assetData(token).sharePremium
```

The Teller applies an owner-settable haircut when minting:

```
// TellerWithYieldStreaming.sol:76-78 (Base V0.3)
shares = depositAmount.mulDivDown(ONE_SHARE, accountant.getRateInQuoteSafe(depositAsset) + 1);
shares = asset.sharePremium > 0 ? shares.mulDivDown(1e4 - asset.sharePremium, 1e4) : shares; // @audit
↳ up to MAX_SHARE_PREMIUM = 1_000 bps
if (shares < minimumMint) revert TellerWithMultiAssetSupport__MinimumMintNotMet();
```

assetData(mUSD).sharePremium is 0 on Monad on the day of this review, but a single updateAssetData call can set it anywhere up to 10%. The premium applies only to minting, so the withdrawal guidance is unaffected.

Recommended Mitigation: Consider updating the Natspec. Also consider adding a deposit test with a non-zero sharePremium:

```
- *   bound. The Veda vault conversion is always at fair value; rate drift from yield streaming
- *   is negligible. A tolerance of 0.1-0.5% is recommended. If this check causes a revert,
- *   no funds are lost - retry with a fresh quote.
+ *   bound. Derive it from the Teller's current mint configuration: `depositAmount * ONE_SHARE /
```

```
+ *      (rateInQuote + 1)`, reduced by `assetData(depositToken).sharePremium` (currently 0, up to 10%),
↳ then
+ *      apply a small tolerance for rate drift between quote and execution. If this check causes a
↳ revert,
+ *      no funds are lost - retry with a fresh quote.
```

Metamask:

Fixed in commit [40cd24a](#).

Cyfrin: Verified.

7.2.7 The documented --skip-simulation workaround does not resolve deployment failure

Description: DeployComplianceVedaAdapter tells a Monad deployer to add --skip-simulation because mUSD uses post-London opcodes:

```
* forge script script/DeployComplianceVedaAdapter.s.sol --rpc-url <rpc_url> --private-key $PRIVATE_KEY
↳ --broadcast
* Monad simulations may require --skip-simulation because its mUSD contract uses post-London opcodes.
```

The repository pins London:

```
evm_version = "london"
```

Foundry 1.7.1 describes --skip-simulation as skipping on-chain simulation. The script still runs locally first. Inside `vm.startBroadcast()`, it creates the adapter. The constructor approves the BoringVault for `depositToken`:

```
// Approve BoringVault to pull tokens
depositToken.forceApprove(boringVault, type(uint256).max);
```

`SafeERC20.forceApprove` calls `approve(spender, value)` and, if that returns false, `approve(spender, 0)` and then `approve` again. On a London fork, mUSD hits an opcode London does not enable. Both `approve` calls return `NotActivated`. The follow-up goes through `Address.functionCall`, which reverts with `FailedInnerCall()` (0x1425ea42) because the failed call has no revert payload. The script stops in that local run. On-chain simulation never starts, so the flag has nothing left to skip.

The script leaves a `vm.mockCall` on that `approve` commented out. The comment above it already names `NotActivated` in `revm`.

Dry run, Foundry 1.7.1 (4072e48705af), <https://rpc.monad.xyz>, chain 143, mUSD 0xacA92E438df0B2401fF60dA7E4337B687a vault 0xBFeC8c2b1ccea3931a1363E4CaC27352c1C908B7. No --broadcast. Owner and delegation manager were non-zero placeholders. The approve target was the live vault and token.

Command	Result
Repository defaults, plus --skip-simulation	Local constructor reverts. Both approve calls return NotActivated, then FailedInnerCall()
Same inputs, plus --evm-version shanghai	Script completes. Foundry prints SKIPPING ON CHAIN SIMULATION.

`forge inspect` creation code for `ComplianceVedaAdapter` is 8172 bytes on London (0x31592e7b6d29d9fc1dace4c3380a422a98) and 7973 bytes on Shanghai (0x12b55a90362e52f37ca38f778781740e4955e03861d5ff1ec4f07fc55d21fdcd). The same salt and constructor arguments hash to different CREATE2 addresses.

`DeployVedaAdapter.s.sol` gives the same --skip-simulation instruction. `VedaAdapter`'s constructor calls `forceApprove` the same way.

Impact: A deployer who follows the documented --skip-simulation workaround still cannot complete the script on Foundry 1.7.1. No transaction was broadcast.

Recommended Mitigation: Document a Foundry EVM version that can execute mUSD, and compile, deploy, and verify with that same setting. `--evm-version shanghai` completed this dry run. A different compiler EVM version changes creation code and the CREATE2 address.

MetaMask: Fixed in [dc5df95](#).

Cyfrin: Verified.

7.2.8 Unclear compliance rejection semantics can lead to duplicate deposits on retry

Description: The adapter's compliance documentation does not clarify that `TellerWithMultiAssetSupport__ComplianceCheckFailed()` can indicate either that an approval's deadline has passed or that its acceptance window has not yet begun. An operator interpreting the latter failure as expiry may issue a replacement approval and fresh leaf delegation without realizing that the original attempt remains executable once its window opens.

The upstream `verifyAndMark` implementation performs the following checks:

```
if (block.timestamp > deadline) {
    revert TellerWithMultiAssetSupport__ComplianceCheckFailed();
}

if (complianceWindow > 0 && deadline > block.timestamp + complianceWindow) {
    revert TellerWithMultiAssetSupport__ComplianceCheckFailed();
}
```

For a nonzero `complianceWindow`, an approval is accepted only when its deadline falls within:

```
block.timestamp <= deadline <= block.timestamp + complianceWindow
```

For a fixed deadline and nonzero `complianceWindow`, the acceptance window begins at `deadline - complianceWindow` and ends at `deadline`, inclusive. Submitting before this window or after it produces the same `TellerWithMultiAssetSupport__ComplianceCheckFailed()` error. The error therefore does not, by itself, establish that an approval has expired. An early rejection leaves the approval unconsumed and rolls back the delegation's transfer-allowance consumption, allowing the original attempt to succeed later.

The Teller's replay key includes the deadline. A replacement approval with a corrected deadline therefore has a different replay key, and consuming it does not consume or cancel the original approval.

For example, assume Alice delegates a total allowance of 1,000 tokens to Bob, who invests 100 tokens at a time, and the Teller uses a one-hour compliance window:

1. At 10:00, Bob submits a 100-token leaf delegation with approval A, whose deadline is 12:00. The deposit fails because the deadline exceeds the permitted future window.
2. At 10:01, Bob prepares a replacement attempt with a fresh 100-token leaf under the same root delegation and approval B, whose deadline is 11:00. The replacement succeeds.
3. At 11:00, approval A enters its acceptance window. The original leaf is still unused, and the root retains sufficient allowance. A party holding the original signed delegation and approval can submit that attempt successfully.

The two attempts execute 200 tokens of deposits for one intended 100-token investment. No additional root authorization from Alice is required.

The compliance guidance in `ComplianceVedaAdapter.sol:30-41` and `231-233` should make this distinction explicit so operators do not treat an early rejection as permanent expiry. This matters when an operator creates a fresh leaf for each replacement attempt. If the operator instead reuses the same correctly amount-capped leaf, the first successful deposit exhausts that leaf and prevents the second execution.

Impact: Retry flows that issue a replacement approval and fresh leaf without invalidating the original attempt can execute the same logical deposit twice. This unintentionally consumes additional delegated allowance and converts more of the user's liquid assets into vault shares than the operator intended for that investment.

Both deposits remain subject to the root's aggregate allowance, and the resulting shares are delivered to the root delegator. The risk applies where that allowance can cover both attempts.

Recommended Mitigation: Document that `TellerWithMultiAssetSupport__ComplianceCheckFailed()` can mean either that the deadline has passed or that the acceptance window has not yet begun. Warn operators that an early rejection leaves the original approval usable later and that issuing a replacement does not invalidate it. Retry guidance should recommend reusing the same amount-capped leaf or revoking the superseded leaf before executing a replacement.

MetaMask: Fixed in [9a8fa2b](#).

Cyfrin: Verified.